Laboratorio di programmazione per sistemi ciberfisici

2. Introduzione alla programmazione

Enrico Martini *Michele Boldo*

A.A. 2026/2027

La programmazione

Obiettivo: esprimere la soluzione a un problema mediante istruzioni eseguibili dalla macchina.

Problema $\rightarrow$ **Algoritmo** $\rightarrow$ **Programma** $\rightarrow$ **Verifica**

Algoritmo e programma

Un **algoritmo** è una sequenza finita di passi precisi per risolvere un problema.

Un **programma** esprime questi passi in un linguaggio di programmazione.

1. **Analisi della consegna**: capire dati e risultato richiesto.
2. **Progettazione della soluzione**: individuare i passi dell'algoritmo.
3. **Implementazione**: tradurre l'algoritmo in istruzioni, per esempio in C.
4. **Test**: confrontare i risultati ottenuti con quelli attesi.

Dal problema al test

1. PROBLEMA

Cosa ottenere?

La somma di due numeri

2. ALGORITMO

Quali passi seguire?

Prendi → Somma → Mostra

3. PROGRAMMA

Traduci i passi in **istruzioni** usando un **linguaggio di programmazione**.

4. TEST

Il risultato coincide?

Dati: **10 e 23** → Atteso: **33**

Cos'è un linguaggio di programmazione?

Un **linguaggio di programmazione** permette di esprimere algoritmi attraverso istruzioni che un computer può eseguire, dopo la traduzione o tramite un interprete.

Sintassi — come si scrive

Le regole per costruire istruzioni valide: simboli, parole e ordine degli elementi.

Semantica — cosa significa

Il significato delle istruzioni: quali operazioni descrivono e quali effetti producono.

Esempi: C, Python, Java.

Uno stesso algoritmo può essere espresso in **linguaggi diversi**.

Linguaggi di programmazione

- **Macchina**: istruzioni binarie eseguite dal processore.
- **Assembly**: istruzioni simboliche legate all'architettura.
- **Alto livello**: costrutti più astratti, come in C e Python.

Stampare un messaggio: due frammenti

Assembly — Linux, x86 a 32 bit

```
1 mov eax, 4
2 mov ebx, 1
3 mov ecx, messaggio
4 mov edx, 13
5 int 0x80
```

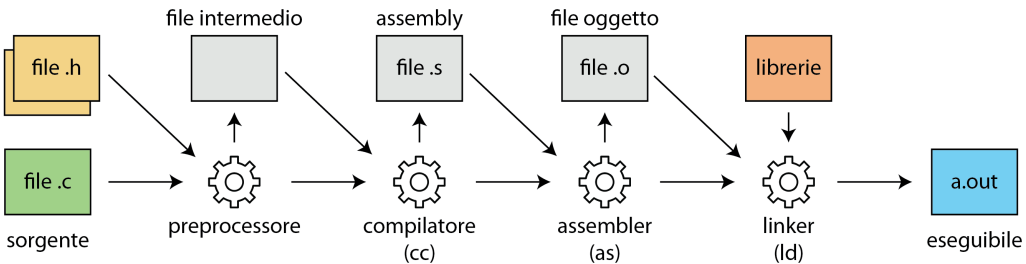
C

```
1 printf("Hello World!\n");
```

Il codice C viene **compilato** prima dell'esecuzione.

Compilazione di un programma

Esistono particolari programmi chiamati **compilatori** che traducono le istruzioni di un linguaggio ad alto livello in istruzioni di basso livello.



Compilazione nel dettaglio

`.c` → `.i` → `.s` → `.o` → **eseguibile**

- **Preprocessore**: elabora direttive come `#include`.
- **Compilatore**: traduce il codice C preprocessato in assembly.
- **Assemblatore**: produce il codice oggetto.
- **Linker**: combina file oggetto e librerie per ottenere l'eseguibile.

Un solo comando coordina normalmente queste fasi:

```
1 gcc programma.c -o programma
```

Primo programma C

Il **codice sorgente** è il testo del programma, salvato in un file.

File sorgente: `hello_world.c`

```c
#include <stdio.h>

int main(void) {
    printf("Hello world\n");
    return 0;
}
```

Output

```
Hello world
```

Compilare ed eseguire

Dalla cartella che contiene il file sorgente:

```
gcc hello_world.c -o hello_world
./hello_world
```

- `gcc`: avvia gli strumenti di compilazione.
- `-o hello_world`: assegna il nome all'eseguibile.
- `./hello_world`: esegue il file nella cartella corrente.

Senza `-o`, il nome predefinito è normalmente `a.out` nell'ambiente del laboratorio.

Compilazione e correttezza

Se la compilazione termina con successo, viene prodotto l'eseguibile.
La **correttezza del risultato** va verificata separatamente.

Modifica, salva, ricompila, esegui

L'eseguibile non si aggiorna quando modifichi il sorgente.

1. Modifica il file `hello_world.c`.
2. Salva il file.
3. Ricompila:

```
1 gcc hello_world.c -o hello_world
```

Se la compilazione riesce, esegui:

```
1 ./hello_world
```

Stai eseguendo la versione aggiornata?

Senza ricompilare esegui ancora la versione precedente. Se la compilazione fallisce, un vecchio eseguibile potrebbe essere ancora presente.

Attenzione al nome dopo `-o`

`-o` indica il nome del file di **output**:

```
1 gcc hello_world.c -o hello_world
```

- `hello_world.c`: sorgente da leggere.
- `hello_world`: eseguibile da creare.

Controlla il nome dell'output

Un file già esistente con quel nome può essere sovrascritto. Usa un nome diverso dal sorgente e dagli altri file che vuoi conservare.

Per l'eseguibile usa, per esempio, `hello_world`, senza l'estensione `.c`.

Errori e avvisi

- **Errore di compilazione**: impedisce di compilare.
  Esempio: manca `;` dopo `printf(...)`.
- **Warning**: segnala un problema potenziale,
  ma normalmente permette di compilare.
- **Errore logico**: il programma produce un risultato errato.
  Esempio: sottrae invece di sommare.

Abilitare più avvisi

```
1 gcc -Wall -Wextra hello_world.c -o hello_world
```

Compilare non basta

Un programma può compilare senza errori e produrre un risultato sbagliato.

Leggere un errore del compilatore

Nel sorgente manca il punto e virgola:

```
4     printf("Hello world\n")
5     return 0;
```

Esempio di messaggio, con Clang:

```
1 hello_world.c:4:28: error: expected ';' after expression
```

- `hello_world.c`: file da controllare.
- `4:28`: riga e colonna segnalate.
- `error`: errore che impedisce la compilazione.
- `expected ';'`: il compilatore si aspettava un punto e virgola.

La posizione indica dove il problema è stato rilevato: la causa può trovarsi anche nella riga precedente.

Correggere gli errori, un passo alla volta

1. Leggi il **primo errore** segnalato.
2. Apri il file alla riga indicata.
3. Controlla anche le righe precedenti.
4. Correggi, salva e ricompila.

Nell'esempio:

```
1     printf("Hello world\n");
2     return 0;
```

Molti messaggi, una sola causa

Un errore può provocare altri messaggi nelle righe successive. Dopo la prima correzione, alcuni potrebbero scomparire.

Leggi anche i **warning**: la compilazione può riuscire anche quando il codice contiene un problema.

Analisi del primo programma

```c
#include <stdio.h>  // Header che dichiara printf

int main(void) {    // Funzione iniziale, senza parametri
    printf("Hello world\n"); // Stampa sull'output standard
    return 0;       // Termina main segnalando successo
}                   // Le graffe delimitano il corpo
```

- `;` termina dichiarazioni, assegnamenti, chiamate a funzione e `return`. Non va dopo `#include`.
- Maiuscole e minuscole sono distinte.
- `//` introduce un commento fino a fine riga.
- `\n` produce un ritorno a capo.

L'**indentazione** rende visibile la struttura, ma non sostituisce le graffe.

I commenti

I commenti spiegano il codice a chi lo legge: non sono istruzioni eseguite dal programma.

```
1 // Commento fino alla fine della riga
2 printf("Hello world\n"); // Stampa un messaggio
3
4 /* Commento che può
5    occupare più righe */
```

- `//`: dalla sua posizione fino a fine riga.
- `/* ... */`: tra apertura e chiusura.
- I commenti `/* ... */` non si possono annidare.

Usa i commenti per chiarire lo scopo o il motivo di una scelta, senza ripetere ciò che il codice dice già.

La funzione printf()

Dichiarata in `stdio.h`. Il primo argomento è la **stringa di formato**.

- Segnaposto e valori corrispondono **nell'ordine**.
- I tipi devono essere **compatibili**.
- Le espressioni sono calcolate **prima della stampa**.

```c
printf("Ciao Mondo\n");
// Ciao Mondo

printf("L'albero ha %i mele.\n", 5);
// L'albero ha 5 mele.

printf("%i + %i = %i\n", 12, 15, 12 + 15);
// 12 + 15 = 27
```

Testo e valori

Dentro le virgolette, `12 + 15` è **testo**.

```
1 printf("12 + 15\n");
```

Output:

```
1 12 + 15
```

Fuori dalle virgolette, `12 + 15` è un'**espressione da calcolare**.

```
1 printf("%i\n", 12 + 15);
```

Output:

```
1 27
```

Le variabili

Una **variabile** è una zona di memoria identificata da un **nome** e un **tipo**.

All'interno di `main`:

```
1 int a;      // Dichiarazione: nome e tipo
2 a = 5;      // Assegnamento: memorizza un valore
3 int b = 3; // Inizializzazione: valore iniziale
```

Il **tipo** determina quali valori la variabile può rappresentare.

Prima di leggere una variabile

Le variabili locali come `a` non partono automaticamente da zero: occorre assegnare loro un valore.

L'assegnamento cambia lo stato

```c
int a = 5;
int b = a;
a = 8;
printf("%i %i\n", a, b); // Output: 8 5
```

`b` riceve il valore attuale di `a`: non si crea un collegamento permanente.

```c
a = a + 1; // a passa da 8 a 9
```

Si calcola il lato destro e si memorizza il risultato nella variabile a sinistra.

In C, = indica un **assegnamento**, non un'uguaglianza matematica.

Come chiamare le variabili

Ogni variabile:

- Deve iniziare per **lettera o underscore** (`var, _variabile, A , b, ...`)
- Deve contenere una sequenza di **caratteri alfa-numerici** (`var1, var2, ...`)
- E' **case-sensitive** (`var` $\neq$ `Var`)
- Non può avere parole **riservate** (no `int, main, return , ...`)
- Non può contenere **spazi** (no `variabile 1, A B, ...`)
- Deve avere un **nome rappresentativo** (no `pippo, pluto, ...`)

I Tipi Fondamentali

Keyword	Descrizione	Segnaposto
`int`	Numero intero	`%i`
`float`	Numero decimale a precisione singola	`%f`
`double`	Numero decimale a precisione doppia	`%lf`
`char`	Singolo carattere	`%c`
`_Bool`	Valore booleano (0,1)	`%i`

```c
int    x = 10;              // numeri interi: 1, 2, -10, 1243
float  y = 0.2;             // numeri decimali: 0.1, 145.2 , -2353.27
double z = 0.21342342;      // numeri decimali molto piccoli: 0.00000000012
char   w = 'm';             // caratteri della tabella ASCII: 'a', 'Z', '9'
_Bool  v = 0;               // numero che corrisponde a VERO (1) o FALSO (0)
```

In `printf`, `%d` e `%i` sono equivalenti: stampano un intero in base decimale.

Tabella ASCII

```
cook@pop-os:~$ ascii -d
   0 NUL     16 DLE     32        48 0     64 @     80 P     96 `    112 p
   1 SOH     17 DC1     33 !      49 1     65 A     81 Q     97 a    113 q
   2 STX     18 DC2     34 "      50 2     66 B     82 R     98 b    114 r
   3 ETX     19 DC3     35 #      51 3     67 C     83 S     99 c    115 s
   4 EOT     20 DC4     36 $      52 4     68 D     84 T    100 d    116 t
   5 ENQ     21 NAK     37 %      53 5     69 E     85 U    101 e    117 u
   6 ACK     22 SYN     38 &      54 6     70 F     86 V    102 f    118 v
   7 BEL     23 ETB     39 '      55 7     71 G     87 W    103 g    119 w
   8 BS      24 CAN     40 (      56 8     72 H     88 X    104 h    120 x
   9 HT      25 EM      41 )      57 9     73 I     89 Y    105 i    121 y
  10 LF      26 SUB     42 *      58 :     74 J     90 Z    106 j    122 z
  11 VT      27 ESC     43 +      59 ;     75 K     91 [    107 k    123 {
  12 FF      28 FS      44 ,      60 <     76 L     92 \    108 l    124 |
  13 CR      29 GS      45 -      61 =     77 M     93 ]    109 m    125 }
  14 SO      30 RS      46 .      62 >     78 N     94 ^    110 n    126 ~
  15 SI      31 US      47 /      63 ?     79 O     95 _    111 o    127 DEL
```

Caratteri e codici ASCII

Un carattere viene rappresentato mediante un **codice numerico**. In ASCII, alla lettera `A` corrisponde il codice `65`.

```c
char lettera = 'A';

printf("%c\n", lettera); // A
printf("%i\n", lettera); // 65 in ASCII
```

`%c` mostra il **carattere**; `%i` mostra il **valore numerico**.

Limiti di ASCII

ASCII comprende 128 codici e non rappresenta tutti i caratteri, come le lettere accentate italiane.

Esercizio 1

Compilare ed eseguire il seguente programma:

```c
#include <stdio.h>

int main(void) {
  printf("Hello world\n");
  return 0;
};
```

Esercizio 2

Compilare ed eseguire il programma che calcola la somma di due numeri interi e stampa a video il risultato:

```c
#include <stdio.h>

int main(void) {
  int value1;
  int value2;
  int sum;
  value1 = 10;
  value2 = 23;
  sum = value1 + value2;
  printf("La somma di %i e %i e':%i\n", value1, value2, sum);
  return 0;
};
```

Esercizio 3

Scrivere un programma che calcola la differenza tra `0.12357439` e `24.962816` e stampa a video il risultato.

Esercizio 4

Scrivere un programma che stampi a video la tabellina del 43 (usare $*$ per la moltiplicazione).

```
43 x 1 = 43
43 x 2 = ...
43 x 3 = ...
43 x 4 = ...
43 x 5 = ...
43 x 6 = ...
43 x 7 = ...
43 x 8 = ...
43 x 9 = ...
43 x 10 = 430
```

Esercizio 5

Scrivere un programma che dichiari 4 variabili A,B,C,D che dovranno contenere numeri decimali. Assegnare alle variabili, rispettivamente, i valori: 12.3, 15.5321, -93.0 e 12746.0372. Stampare a video il risultato della somma di A con B e della differenza tra D e C.

```
A + B = ...
D - C = ...
```

Esercizio 6

Svolgere gli esercizi presenti su Moodle, alla sezione ”Esercizi”, dal titolo ”Introduzione”.